ECSE 444 - Microprocessors
Final Project - Final Report

Antoine Phan 260948633 | Camille Pouliot 261114809 | Lena Bousserhane 261108552 | Noorjahan Abonty 260953393

Department of Electrical and Computer Engineering
McGill University

I. INTRODUCTION

For our project, our team considered several gameplay ideas at the start of the development, including calibration mode, difficulty settings and a competitive two-player option. These early ideas were really helpful to define the scope of our project but the final system focused on implementing the essential technical components : tone synthesis, microphone acquisition, and pitch estimation. This helped us to focus more on hardware configuration and digital-signal-processing methods rather than game mechanics.

It uses a simple state machine to coordinate the user interface, audio generation, recording and analysis. The STM32's built-in DAC, DFSDM microphone interface, DMA channels, and CMSIS-DSP library enable real-time operations on a lightweight embedded platform. By combining these elements, the system demonstrates how audio-based interaction can be achieved entirely on a microcontroller without external processing.

II. SYSTEM OVERVIEW

Our game system is structured around several interacting components that together implement the full game functionality. Figure 1 represents a block diagram showing how audio output, microphone input, digital signal processing, and the computer terminal interface communicate through the STM32 B-L475E-IOT01A board. The speaker is driven by the board's DAC to produce the reference tone, while the onboard DFSDM microphone captures the user's response. The recorded signal is then passed to the CMSIS-DSP processing block for frequency estimation, and the results are communicated to the user through UART. A FSM oversees these interactions and determines the transition between playback, recording, analysis and display.

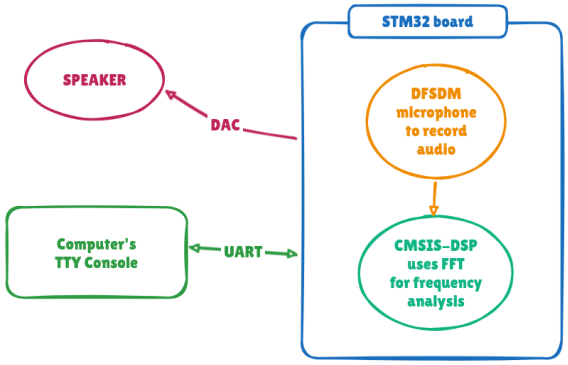


Figure 1: System Architecture of the Pitch-Matching Game.

II. A. SYSTEM COMPONENT AND DATA FLOW

As shown in Figure 1, the system is composed of the following elements:

- **DAC Speaker:** Plays the reference/target tone generated by the STM32 through the DAC.
- **DFSDM Microphone:** Captures the user's singing response.
- **CMSIS-DSP Processing Block:** Performs FFT-based frequency estimation on the recorded audio.
- **UART Console:** Provides a simple text-based user interface for starting the game, prompting the user, and displaying accuracy results
- **STM32 Board:** Coordinates all peripherals and runs the finite-state machine that drives the game.

This block-level representation is very helpful to highlight the main functional blocks of the system and their interactions, keeping the focus on the overall architecture rather than low-level configuration details.

II. B. GAME CONTROL FLOW

To coordinate the different stages of our game, we designed a finite-state-machine that guides the system

from tone playback to recording, analysis and finally the result display. The states are:

- **MENU:** Waits for the user to start the game
- **PLAY_TONE:** Outputs a randomly generated target frequency to the speaker.
- **WAIT_FOR_RECORD:** Prompts the user to sing and press the button to begin recording
- **RECORD:** Captures microphone audio for approximately 2.5 seconds.
- **ANALYZE:** Processes the recorded signal with an FFT and extracts the dominant frequency.
- **SHOW_RESULTS:** Displays the detected pitch, accuracy, and optional feedback on the UART console.

This state based structure keeps each task isolated and prevents tone generation, recording and processing from interfering with one another. This ensures that the gameplay is predictable and responsive.

III. HARDWARE CONFIGURATION

The pitch matching game relies on three hardware subsystems of the STM32 B-L475E-IOT01A board: the DAC for tone generation, the DFSDM interface for audio capture, and the UART module for the text-based interface. All peripherals operate under DMA control to reduce CPU load and allow real-time processing.

III. A. DAC OUTPUT AND TIMER CONFIGURATION

The reference tone is generated using DAC1 Channel 1 driven by TIM2. A 30-sample sine lookup table is streamed continuously through the DAC using circular DMA mode, allowing the tone to play without CPU intervention. Only the timer's auto-reload register determines the output frequency, which reduces computation overhead and enables rapid frequency updates during gameplay.

The timer's TRGO event triggers each DAC conversion, and the effective tone frequency is computed from:

$$f_{tone} = N_{samples}/f_{timer}$$

where $N_{samples}$ is the size of the sine table. The chosen frequency range of 200-1200 Hz is in the region where the DFSDM microphone data is less prone to noise and aligns with the human vocal frequency range. This improves the playback clarity and more importantly the pitch detection's accuracy.

The DAC output is routed to an external speaker through a 165Ω resistor, as per earlier labs.

III. B. DFSDM MICROPHONE INPUT AND DMA CAPTURE

The user's voice is captured through the on-board digital MEMS microphone, interfaced using DFSDM Channel 2 and Filter 0. DFSDM is configured in continuous conversion mode, with DMA writing directly into a buffer holding approximately 2.5 seconds of audio at roughly 44.1 kHz. This recording window helps the player to have sufficient time to stabilize their pitch while staying within the available RAM.

Raw DFSDM samples are 24-bit signed values; in the DMA completion callback, they are shifted, clipped, and scaled to:

- prevent overflow when replayed through the DAC
- normalize the dynamic range for FFT processing
- suppress extreme outlier values due to handling noise.

Empirical testing showed that the vocal recordings fall within roughly -1000 and +2500 counts, and these limits were used for clipping. Converting to 12-bit DAC compatible values allows optional audio playback for debugging. This hardware configuration ensures that audio sampling proceeds uninterrupted during gameplay, with minimal CPU overhead.

III. C. UART CONSOLE INTERFACE

All game interaction occurs through USART1, configured at 115200 baud. UART uses interrupt-driven reception so characters are captured asynchronously while the system performs other tasks. Incoming bytes are stored in a small line buffer until a newline character triggers command parsing. This mechanism keeps user input responsive even during computation-heavy operations.

We tried to optimize transmission functions during development by replacing some HAL-level calls with shorter inline routines. Benchmarking showed a reduction of roughly 400 000 CPU cycles when sending large volumes of text, and an ARM-optimized receive callback executed about 150 000 fewer cycles than the baseline implementation. Although these improvements are only beneficial for smoother console updates, especially when printing multi-line ASCII screens, then actual gameplay.

III. C. PERIPHERAL SYNCHRONIZATION AND CPU LOAD MANAGEMENT

DMA is used for both DAC output and DFSDM input, while UART relies on interrupts. This allows the CPU to remain free for FFT computation during the ANALYZE state. The FSM ensures that playback, recording and processing never takes place simultaneously which will eventually prevent timing conflicts and guarantee deterministic execution.

IV. SIGNAL PROCESSING AND PITCH DETECTION

The system estimates the user's sung pitch entirely on-board using CMSIS-DSP. The processing pipeline consists of four main steps: preprocessing to clean the microphone signal, windowing and FFT computation, multi-peak extraction with interpolation and harmonic scoring method that identifies the most likely fundamental. This section outlines the pitch-detection process and shows how each mathematical step contributes to making the algorithm accurate and reliable on the STM32.

IV.A . PREPROCESSING

DFSDM output contains DC bias and significant low-frequency noise (about 80-180 Hz). Both effects distort the FFT by producing strong low-frequency bins unrelated to the user's voice. To correct this, the average of the recorded frame is subtracted from each sample to remove the DC component. Then we set the cut-off frequency to be 200Hz (set to be above noise) and calculate coefficient α ; $f_{cut\text{-}off} = (1\text{-}\alpha) \cdot F_{sampling}/2\pi \Rightarrow \alpha \approx 0.97$. first-order IIR high-pass filter with is then applied twice:

$$y[n] = \alpha\big(y[n-1] + x[n] - x[n-1]\big)$$

This cascaded structure preserves vocal harmonics all while aggressively removing low-frequency rumble before proceeding to the FFT.

IV.B . WINDOWING AND FFT COMPUTATION

To reduce spectral leakage, the filtered signal is multiplied by a hamming window:

$$w(n) = 0.54 - 0.46\cos\left(\frac{2\pi n}{M-1}\right) \text{ for } 0 \le n \le M-1$$

Before computing a 1024-point real FFT. With a sampling rate of 44.1 kHz, each FFT bin represents:

Frequency = bin_number × (44100 / 1024) ≈ bin_number × 43Hz

Although this resolution is coarse, later interpolation provides sub-bin precision. Only the 200–1200 Hz band is evaluated, matching both the DAC-generated tones and typical vocal fundamentals.

IV.C . MULTI-PEAK DETECTION AND PARABOLIC INTERPOLATION

To achieve sub-bin frequency accuracy beyond the 43 Hz resolution of the FFT, the system refines each peak using parabolic interpolation applied to the magnitudes of three adjacent bins. If the FFT identifies bin i as a candidate peak, then the bins at indices i-1 and i+1 are denoted as y0, y1 and y2 respectively. The interpolation offset δ, which estimates how far the true peak lies between FFT bins, is computed using the standard formula provided in the course notes:

$$\delta = \frac{1}{2}\frac{y_2 - y_0}{2y_1 - y_2 - y_0}$$

This value modifies the estimated peak location from:

$$f_{\text{peak}} = (i + \delta)\left(\frac{44100}{1024}\right)$$

where the term (44100/1024) corresponds to the frequency represented by each FFT bin (approximately 43 Hz). Using this method helped pitch estimation significantly, especially for vocal signals where true sinusoidal peaks can fall between discrete FFT bins.

IV.D . HARMONIC SCORING

Human voices often produce harmonics that are louder than the true fundamental. For example, if a user sings at 200 Hz, the 400 Hz or 600 Hz harmonics may appear stronger in the FFT. To avoid selecting these subharmonics, the algorithm assigns each detected frequency peak a score based on three factors:

- Harmonic reinforcement: peaks that have corresponding peaks near 2×, 3×, or 4× their frequency receive higher scores.
- Subharmonic penalties: if a peak appears to be a subharmonic of a lower fundamental (for example 600 Hz having a subharmonic near 200–300 Hz), its score is reduced.
- Frequency weighting: slightly higher weight is given to mid–high vocal frequencies, reducing the influence of low-frequency noise.

The peak with the strongest harmonic structure and no strong subharmonics is selected as the fundamental. If all peaks fall below a minimum magnitude threshold, the

system returns zero to indicate that the input was too weak or unreliable.

V. GAME FLOW AND UART INTERFACE

The gameplay experience is built around a finite-state machine that structures how the player hears the reference tone, records their voice, and receives feedback through the UART console. Unlike the architectural description in earlier sections, this part focuses on how the system behaves from the player's perspective and how UART enables a smooth interaction loop.

V.A . FINITE-STATE MACHINE

The game progresses through a sequence of states that guide the player through every step. IN MENU, the terminal displays the ASCII title screen and waits for "s" or "start". PLAY_TONE generates a random frequency between 200 and 1200 Hz and plays it through the DAC while showing on-screen instructions. The system then enters WAIT_FOR_RECORD, informing the user to press "r" once ready to sing.

When recording begins, RECORD captures about 2.5 seconds of microphone input using DFSDM with DMA, keeping UART responsive through interrupts. Once the buffer is filled, ANALYZE applies the full DSP pipeline including preprocessing, FFT, multi-peak extraction and harmonic scoring to determine the sung pitch. SHOW_RESULTS prints the target frequency, detected frequency and accuracy rating before returning the user to the menu. This structure ensures that audio playback, recording, and computation remain isolated, this prevents timing interference and ensures consistent behavior during every round.

V.B . UART INTERACTION AND INTEGRATED PLAYER EXPERIENCE

UART at 115200 baud functions as the game's sole interface, transforming a simple terminal window into a lightweight graphical environment through multi-line ASCII screens. Incoming characters are handled asynchronously using interrupt-driven reception, allowing the system to remain responsive even during recording or DSP computation. Characters accumulate in a line buffer until a newline triggers the parser, which recognizes short, intuitive commands such as "s", "start", "r", "record", and "help".

The visual layout of the interface was deliberately designed around a 90 x 24 character screen. A width of 90 provides enough horizontal space for framed menus and ASCII art, and its divisibility by 2, 3, 5, and 10 simplified spacing and alignment. The height of 24 fits standard terminal windows while allowing the display to be cleanly divided into instructional and output sections.

Figure 2 presents the final interface as seen by the player. It illustrates how the UART transmission/receive, structured layout, and state-driven messages are combined to create a clear, intuitive, and fully text-based user experience that supports the overall game flow.



Figure 2: UART - Based User Interface Displayed During Gameplay

VI. PERFORMANCE OPTIMIZATION AND BENCHMARKING

By implementing the built in function for Uart transmission in Arm, we were able to shorten the number of cycles for the transmission by 400 000 when attempting to transmit the string "benchmark" a thousand times.

The same was done for the UART callback function. The number of cycles to read the string "benchmark" a thousand times was reduced by 150 000 cycles or 10% These optimizations do not affect pitch accuracy but improve the overall system smoothness.

VII. CONCLUSION

The final system combines DAC tone generation, DFSDM microphone capture, DMA - driven data transfer, and a CMSIS - DSP pitch detection pipeline to create a complete embedded pitch-matching game on the STM32 B-L475E-IOT01A board. A finite-state machine organizes gameplay, and a UART-based interface provides clear prompts and results without external hardware. Despite running entirely on a microcontroller, the system performs reliable real-time pitch estimation and delivers an intuitive user experience. Possible extensions include note-name detection, multi-round scoring, or alternative pitch-tracking algorithms, but the implemented design meets all project objectives and demonstrates effective use of embedded DSP techniques.